

```

        itinerary2.insert(neighborids3 + j,tempitin[j])
    else:
        itinerary2[neighborids1] = itinerary[neighborids2]
        itinerary2[neighborids2] = itinerary[neighborids1]

    distance1 = howfar(genlines(cities,itinerary))
    distance2 = howfar(genlines(cities,itinerary2))

    itinerarytoreturn = itinerary.copy()

    randomdraw = np.random.rand()
    temperature = 1/((time/1000) + 1)

    if((distance2 > distance1 and (randomdraw) < (temperature)) or (distance1 >
distance2)):
        itinerarytoreturn = itinerary2.copy()

    return(itinerarytoreturn.copy())

```

[Listing 6-5](#): Now, we use several different methods to perturb our itinerary.

Our `perturb()` function is now more complex and more flexible; it can make several different types of changes to itineraries based on random draws. Flexibility is not necessarily a goal worth pursuing for its own sake, and complexity is definitely not. In order to judge whether the complexity and flexibility are worth adding in this case (and in every case), we should check whether they improve performance. This is the nature of tuning: as with tuning a musical instrument, you don't know beforehand exactly how tight a string needs to be—you have to tighten or loosen a little, listen to how it sounds, and adjust. When you test the changes here (shown in bold in [Listing 6-5](#)), you'll be able to see that they do improve performance compared to the code we were running before.

Avoiding Major Setbacks

The whole point of simulated annealing is that we need to do worse in order to do better. However, we want to avoid making changes that leave us *too much* worse off. The way we set up the `perturb()` function, it will accept a worse itinerary any time our random selection is less than the temperature.